Final Project: Generative Recursive Autoencoder for Shape Composition with Substructure Priors

Fuyang Zhang
301415661

April 2020

1 Introduction

Autoencoder is a type of artificial neural network used to learn efficient data codings in an unsupervised manner. The aim of an autoencoder is to learn a representation for an input data. Autoencoder is one of the active tasks in many fields of computer vision, computer graphics, etc. Recent progress on deep learning has stimulated a lot of great works [1–4] on autoencoder in image domain. Compared with 2D image, 3D objects are more challenge in data representation, i.e. 3D shapes do not have a natural parametric structure, while an image can simply be viewed as regular matrix or tensor. Thus, whether a similar success can be achievable in 3D shapes are still a open question.

In this project we are going to learn the inherent challenge of 3D shapes representation. Our work highly refers to this state-of-the-art paper [5] of 3D shape composition, which uses VQ-VAE technique for encoding shapes feature. The code can be found at `https://github.com/zhangfuyang/CMPT764Final`.

2 Recursive Autoencoder for Shape Structure

2.1 Preliminaries

This project tackles the 3D shape feature representation with hierarchical and symmetric structure prior.

Our dataset is from ShapeNet [6], with additional hierarchical information from [7]. Fig. 1 shows the hierarchy and symmetry of the shapes[1].

The first step is data preprocessing. In order to decrease the complexity of data, each pieces of object (such as armrest of a chair) is represented as an oriented bounding box (OBB). A **Tree** Class is defined for storing one shape each (e.g. a chair). In the tree, each node is stored in a **Node** Class, which has three

[1] The picture is adapted from the paper [7]

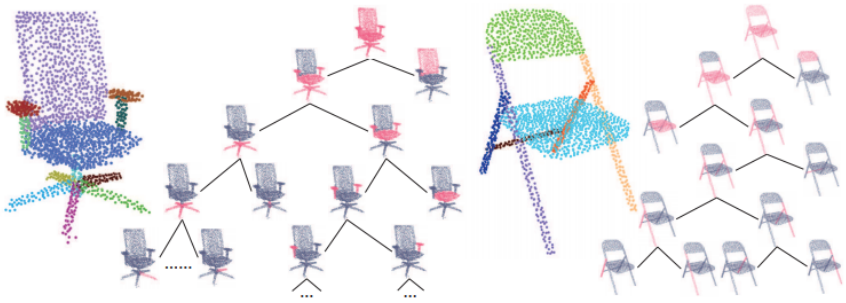


Figure 1: A hierarchy of fine-grained parts for shapes

different types: leaf, adjacency and symmetry. Leaf node and symmetry node contain OBB coordinate and parameters of symmetry respectively. Adjacency node is for merging two children nodes. In high level view, the tree structure is a binary tree structure.

2.2 Overview

The network takes a set of parts as input as will as the the hierarchy structure. The first stage is to recursively encode each node in bottom-up manner. After encoding, the next stage is to train a decoder to decode each node back given the shape structure. A simple overview is shown in Fig 2.

Encoding. The encoder can be divided into 3 sub-networks corresponding 3 different types of node. After network going through the shape, we can get a final fixed size feature for the entire shape.

- 1) Leaf-encoder: input is an oriented bounding box (12 dimensional vector), which is passed to a 3 layer MLP with tanh as nonlinear activation function. The output feature is an 80 dimensional vector.
- 2) Adj-encoder: adj-encoder is used to merge to children node feature, which may come from Leaf node, Symmetry node or Adjacency node. First step is to embed each children feature into a hidden vector (set as 160 dimension in our experiment). Next step, we add those two hidden vector and pass to a 2 layer MLP. The output feature is also an 80 dimensional vector.
- 3) Sym-encoder: sym-encoder is similar to adj-encoder. The first step is to its only child as well as symmetric vector into hidden vectors. Then, add two hidden vectors together and pass to a 2 layer MLP to get a feature vector.

Decoding. The decoder can also be divided into 3 sub-networks which are similar with the encoder's. Therefore, we omit the network details of the decoder to avoid tedious descriptions.

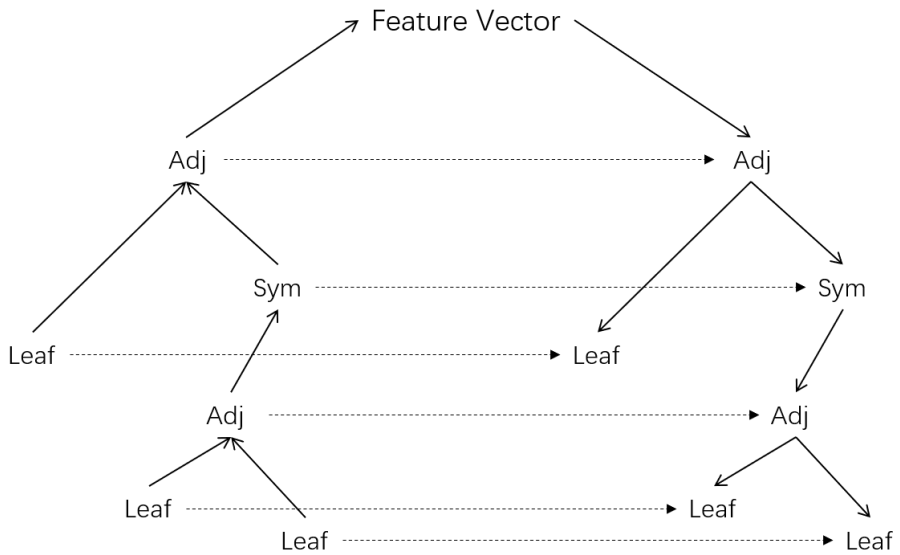


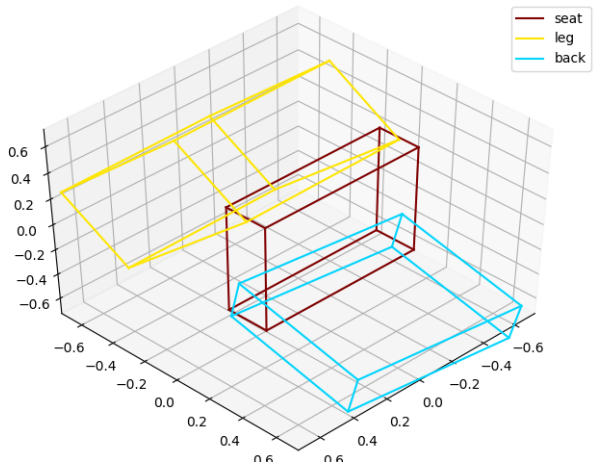
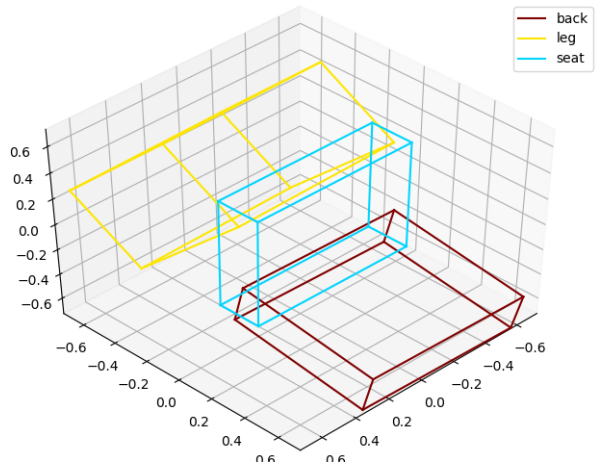
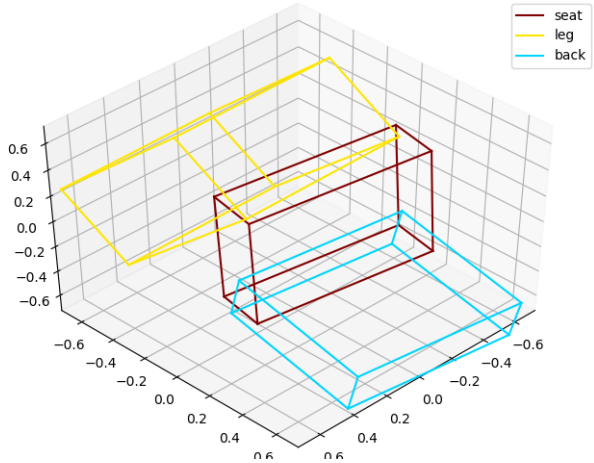
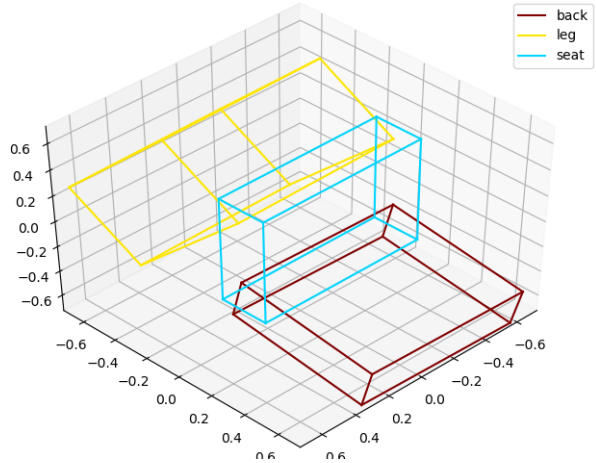
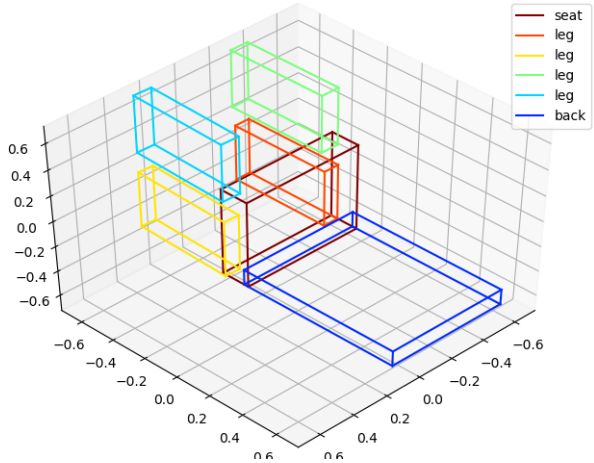
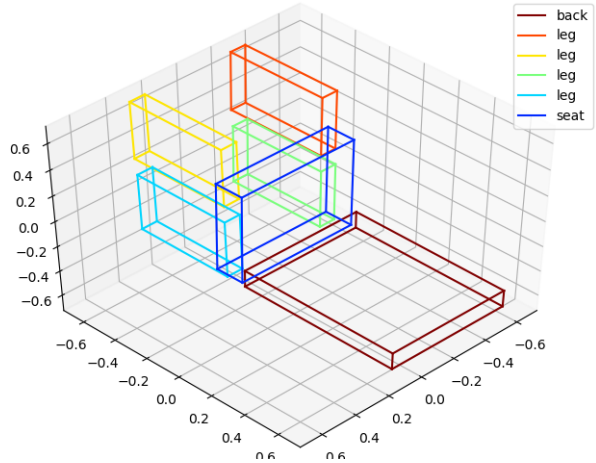
Figure 2: Overview

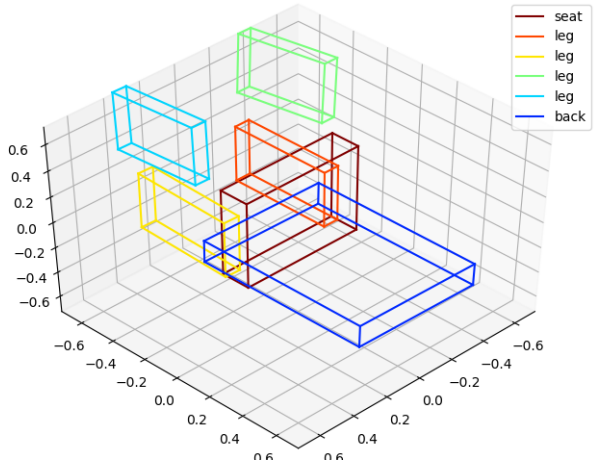
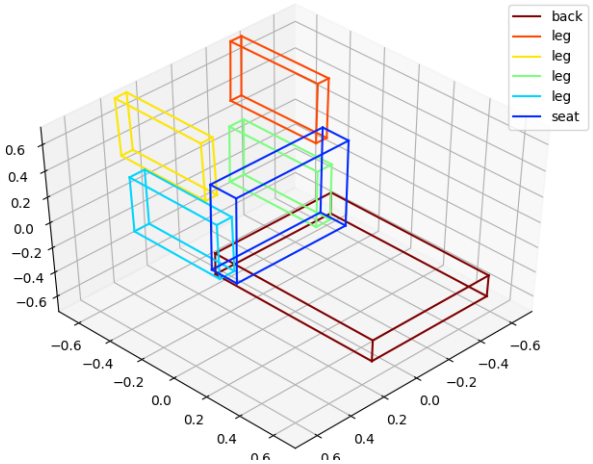
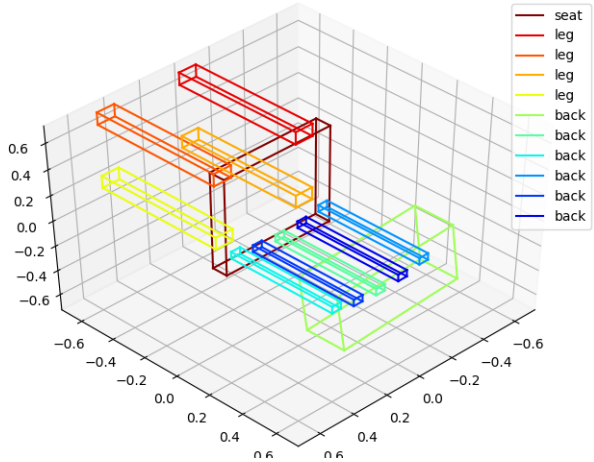
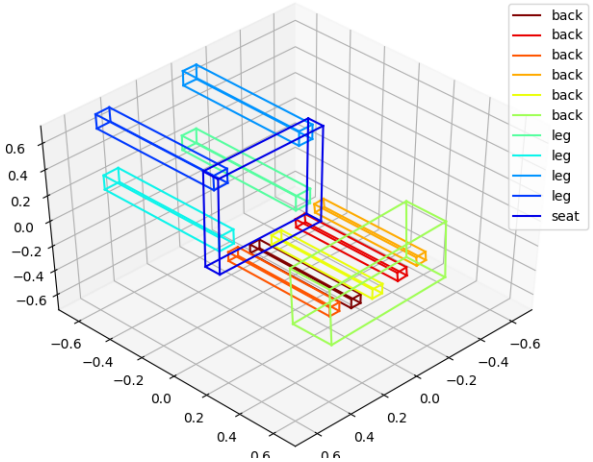
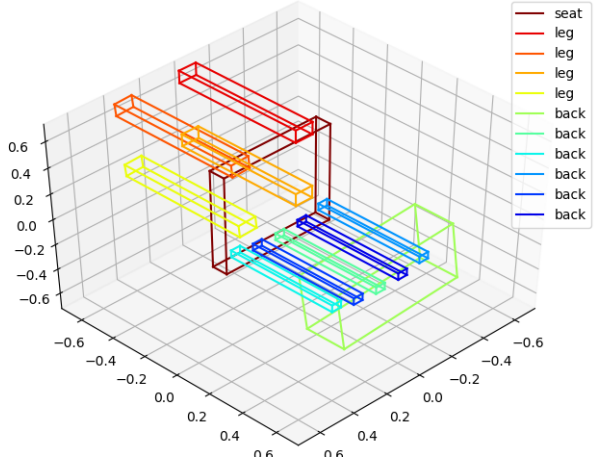
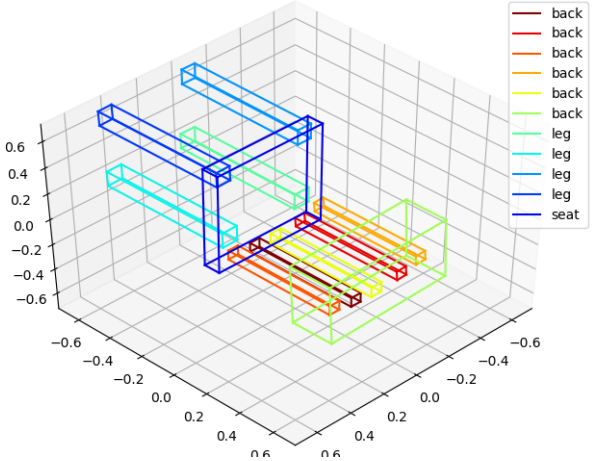
We use VQ-VAE to train the autoencode, which means we also train a discrete latent space codebook. When decoding a feature vector, we replace each decoding feature with nearest Euclidean distance in codebook. In this way, we hope the network can be robust to the noise and should learn a better global alignment than pure autoencoder with local components.

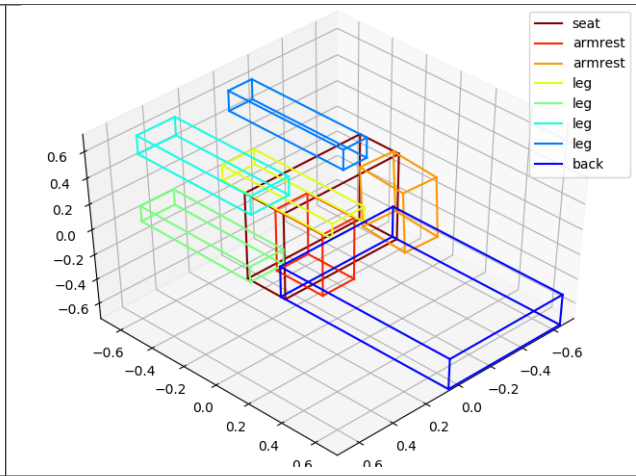
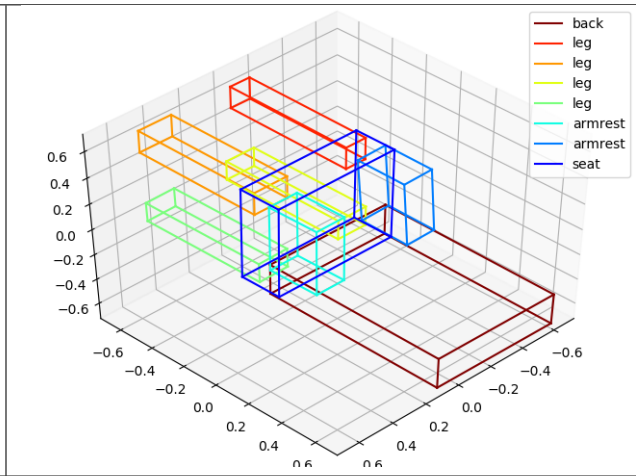
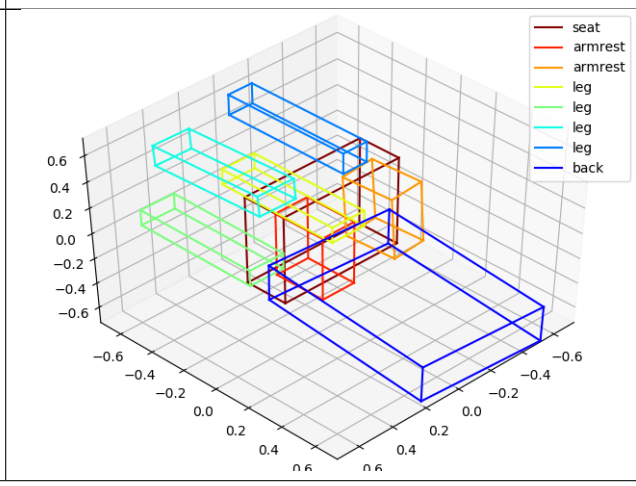
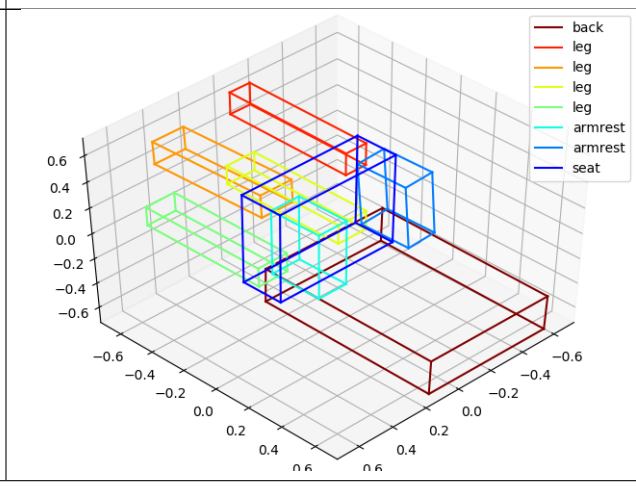
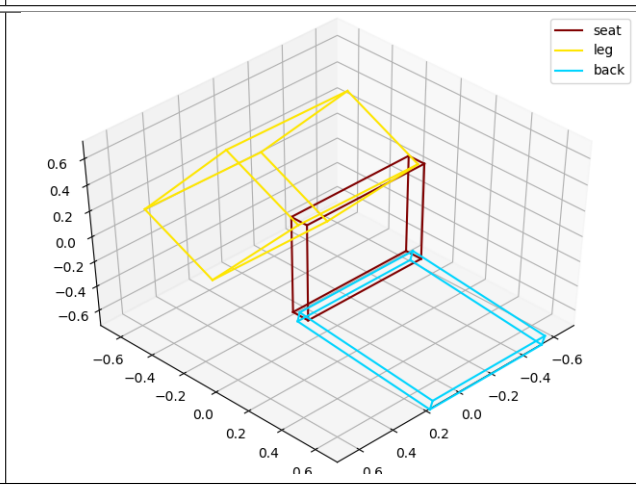
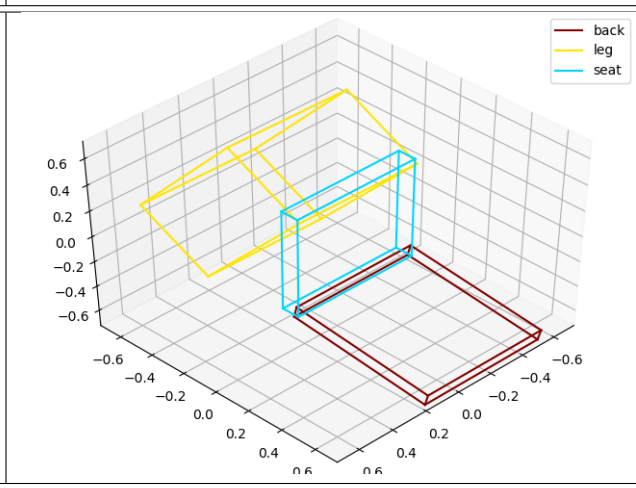
3 Experiment

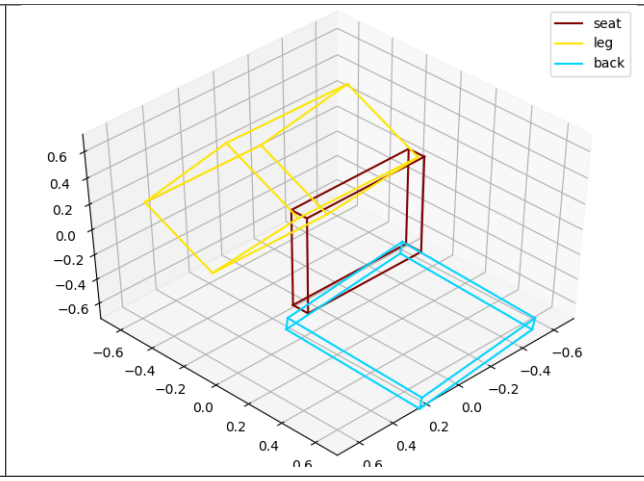
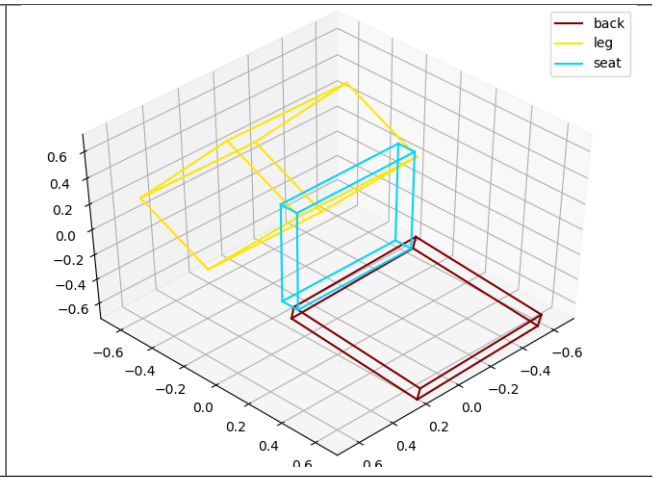
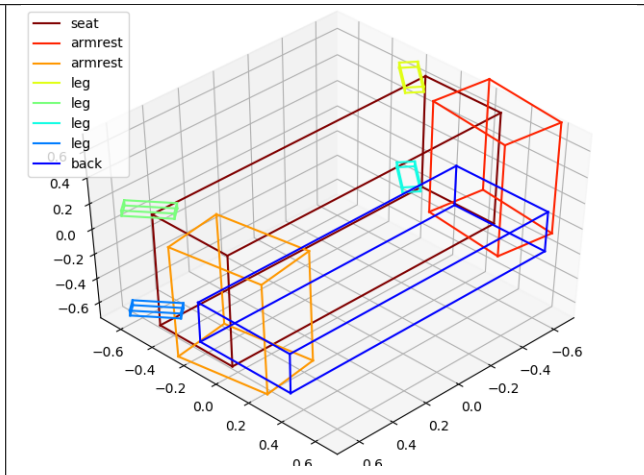
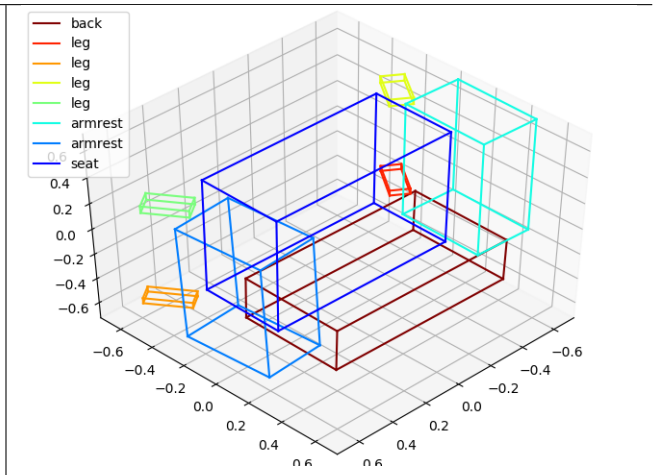
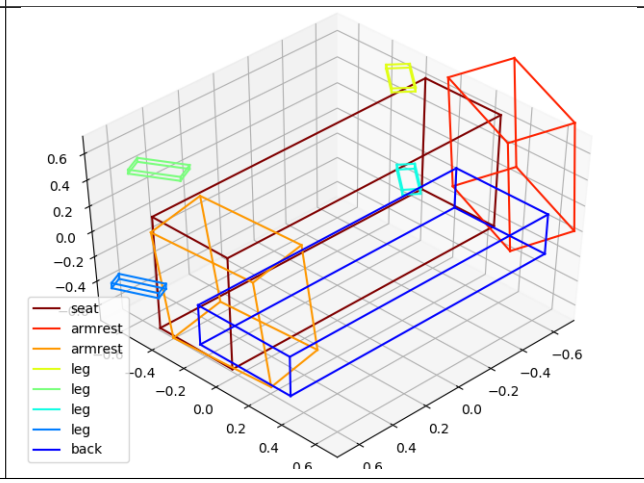
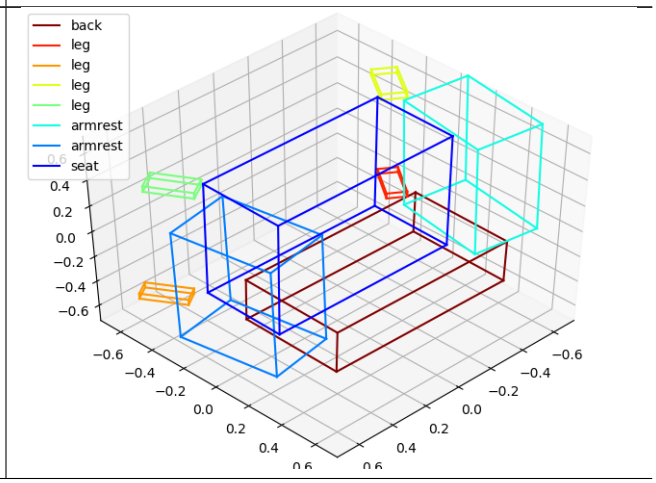
We have implemented the proposed system in PyTorch0.4. The learning rate is initialized to 1×10^{-3} for both encoder and decoder. We used Adam algorithm as our optimizer.

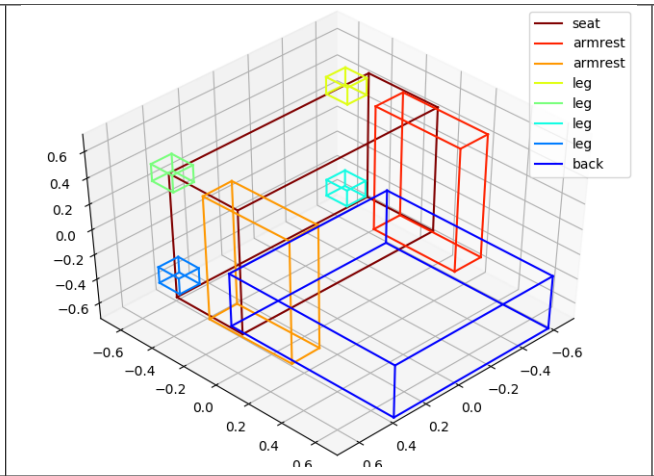
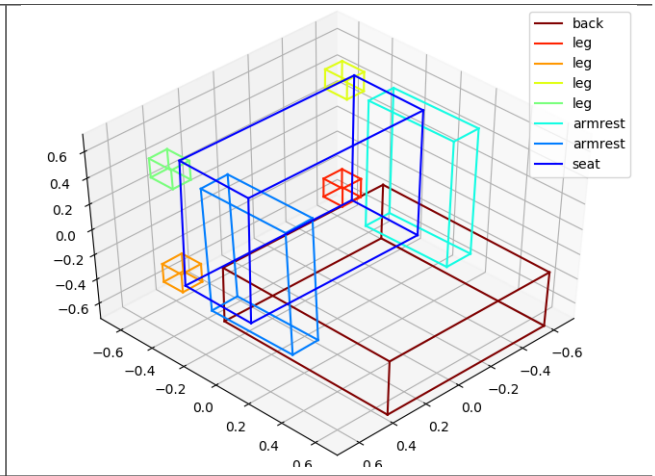
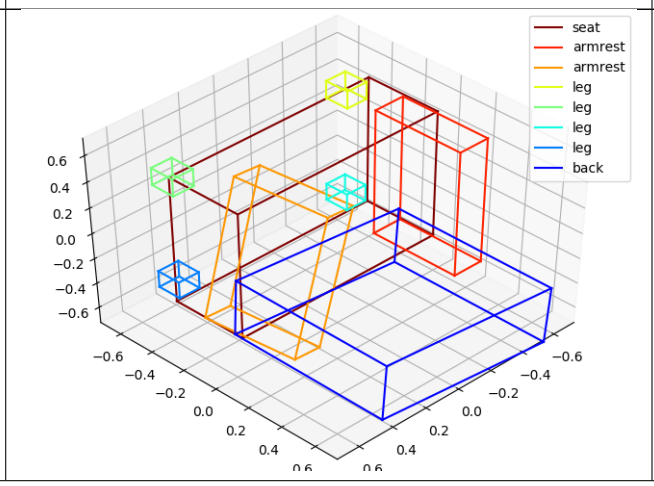
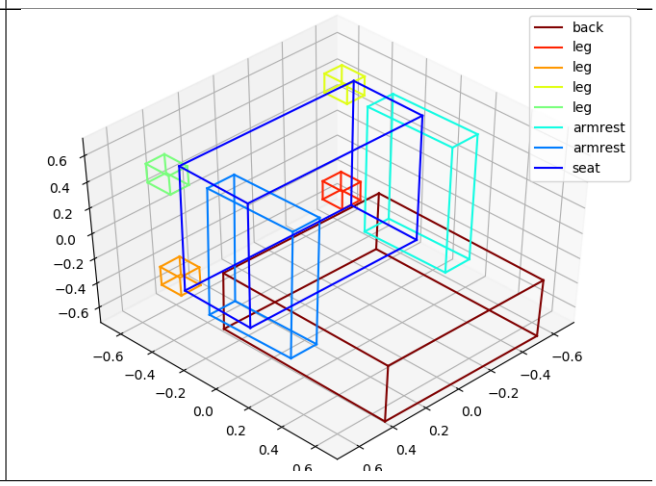
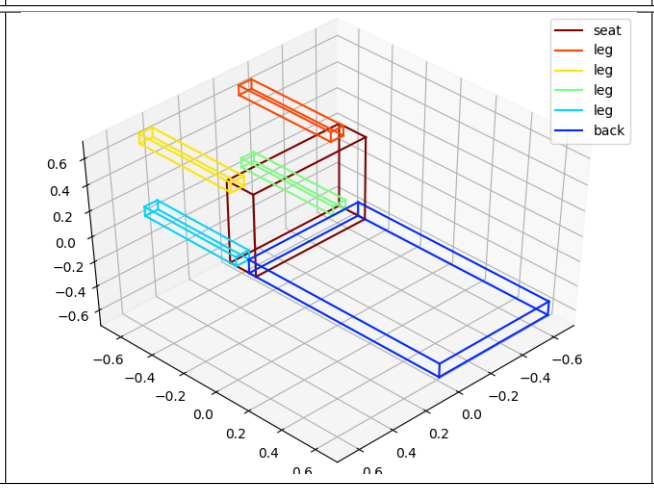
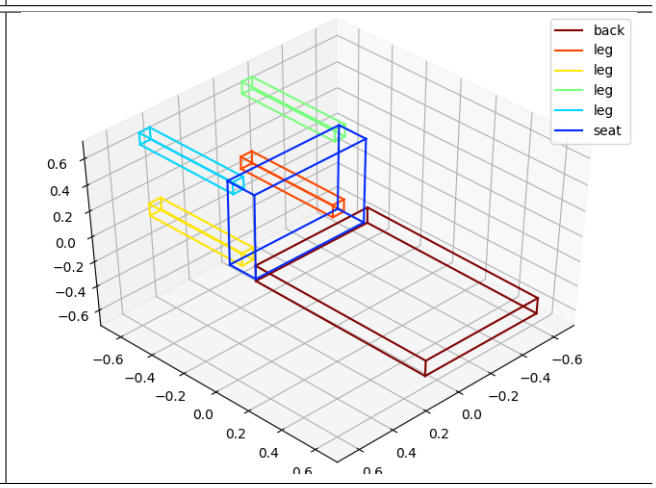
Our training is divided into two procedures. First, we trained whole pipeline with ground truth data without adding noise. We expected codebook can capture the real shape structure feature. Second, we froze codebook, and finetuned the whole network with noisy data. We randomly added noise to the bounding box or symmetry feature in the ground truth data. In this way, we believe our network is able to recover the real location of the shapes based on the clean codebook. The results are shown below.

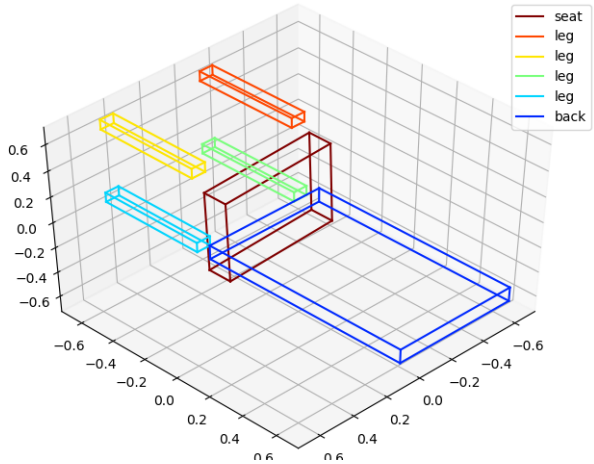
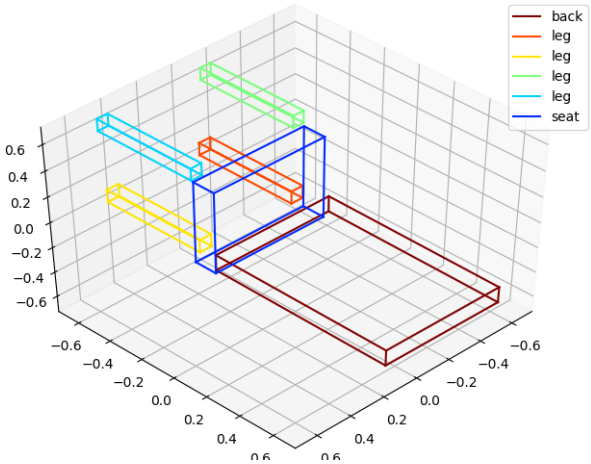
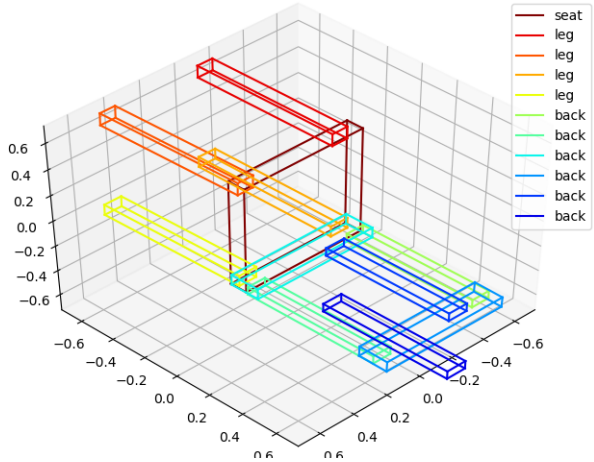
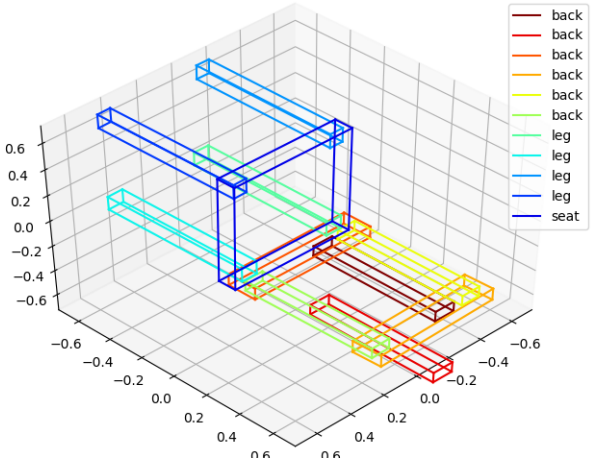
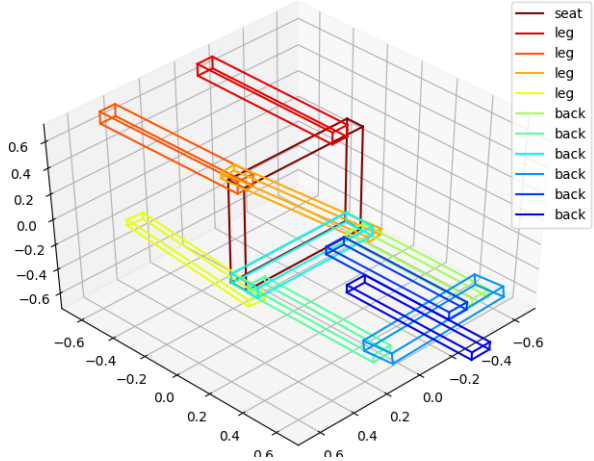
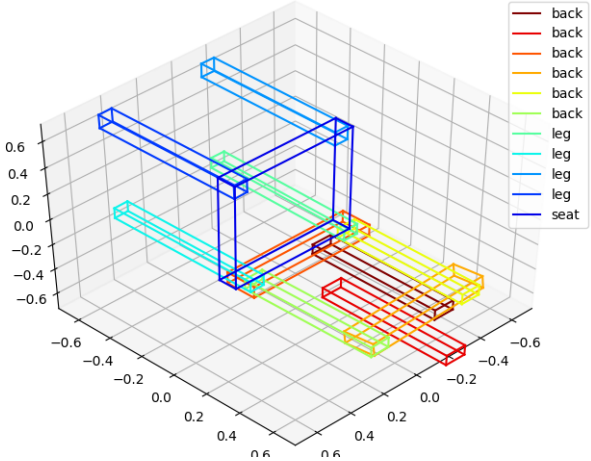
	Ground Truth	Result
w/o noise		
w/ noise		
w/o noise		

w/ noise		
w/o noise		
w/ noise		

w/o noise		
w/ noise		
w/o noise		

w/ noise		
w/o noise		
w/ noise		

w/o noise		
w/ noise		
w/o noise		

w/ noise		
w/o noise		
w/ noise		

4 Future Works

Our work is far beyond finished. The autoencoder is the first step of the work, which solves the shape feature embedding. In order to make our model has the ability of creativity. We should learn how the network encode the feature and what feature distribution codebook has. So our next step is to generate a new shape from the encoder result, might be similar with VQ-VAE-2 [8].

References

[1] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *International Conference on Medical image computing and computer-assisted intervention*. Springer, 2015, pp. 234–241.

[2] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 3431–3440.

[3] I. Gulrajani, K. Kumar, F. Ahmed, A. A. Taiga, F. Visin, D. Vazquez, and A. Courville, "Pixelvae: A latent variable model for natural images," *arXiv preprint arXiv:1611.05013*, 2016.

[4] W. Xu, H. Sun, C. Deng, and Y. Tan, "Variational autoencoder for semi-supervised text classification," in *Thirty-First AAAI Conference on Artificial Intelligence*, 2017.

[5] C. Zhu, K. Xu, S. Chaudhuri, R. Yi, and H. Zhang, "Scores: Shape composition with recursive substructure priors," *ACM Transactions on Graphics (TOG)*, vol. 37, no. 6, pp. 1–14, 2018.

[6] K. Mo, S. Zhu, A. X. Chang, L. Yi, S. Tripathi, L. J. Guibas, and H. Su, "Partnet: A large-scale benchmark for fine-grained and hierarchical part-level 3d object understanding," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 909–918.

[7] F. Yu, K. Liu, Y. Zhang, C. Zhu, and K. Xu, "Partnet: A recursive part decomposition network for fine-grained and hierarchical shape segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 9491–9500.

[8] A. Razavi, A. van den Oord, and O. Vinyals, "Generating diverse high-fidelity images with vq-vae-2," in *Advances in Neural Information Processing Systems*, 2019, pp. 14 837–14 847.